

Home Depot Product Search Relevance

Deep Learning for Semantic Text Matching

1. Introduction & Problem Statement

In e-commerce, accurate search results are critical for user experience. The goal of this project is to predict the **relevance score** (on a scale of 1.0 to 3.0) for a given pair of **Search Term** and **Product Description**.

This is a **Semantic Similarity** problem. A simple keyword match is often insufficient because users may search for "tool" and expect "hammer," or search for "bracket" and find "brace." Our solution explores three increasing levels of complexity to solve this:

1. **Character-Level Siamese Network:** To handle typos and exact string matching.
2. **Word-Level Siamese Network:** Using custom Word2Vec embeddings to capture domain-specific semantics.
3. **Transformer (BERT) Feature Extraction:** Leveraging state-of-the-art pre-trained language models for contextual understanding.

We benchmark all models against a naive **TF-IDF + Ridge Regression** baseline.

Translation-Based Data Augmentation

To reduce overfitting and improve generalization, we applied translation-based data augmentation to the training data. Each training example was augmented by translating the textual fields from English into multiple additional languages (Spanish, French and German). This effectively increased the size of the training set while preserving the original semantic meaning of each query–product pair.

2. Character-Level Modeling

Why Character-Level?

Search queries often contain typos, misspellings, or variations of product codes (e.g., "90deg" vs "90 degree"). Character-level models process text one letter at a time, making them highly robust to these variations without requiring a massive vocabulary of words.

Methodology: Siamese LSTM

We implement a **Siamese Network**, a special architecture designed for comparing two inputs:

1. **Shared Encoder:** Both the *Search Term* and *Product Description* are fed into the *same* LSTM layer (sharing weights). This projects both texts into the same vector space.
2. **Distance Calculation:** We calculate the absolute difference ($|u - v|$) between the query vector and product vector.
3. **Regression Head:** A dense neural network takes this difference and predicts the relevance score.

Benchmark: Naive Statistical Model

To understand if our deep learning models are actually adding value, we create a "naive" baseline. We use **TF-IDF (Term Frequency-Inverse Document Frequency)** to turn text into sparse vectors based on word counts, followed by a simple **Ridge Regression**. This represents a "keyword matching" approach without any semantic understanding.

3. Word-Level Semantic Modeling

Custom Tokenization Strategy

Standard English tokenizers often split important hardware specifications incorrectly (e.g., splitting "90°" into "90" and "°"). To preserve technical meaning, we designed a custom **Regex Tokenizer** that:

- Preserves units attached to numbers (e.g., "90°", "1/2in").
- Keeps special identifiers intact (e.g., "#SC").

Word2Vec Embeddings

Instead of using generic pre-trained embeddings (like GloVe), we train a custom **Word2Vec** model on the Home Depot corpus. This ensures that our model understands that "bracket" and "brace" are semantically close in *this specific context*, rather than just in general English.

Word-Level Siamese Network

We adapt the Siamese architecture from Section 1 to use our custom **Word2Vec embeddings**.

- **Input:** Sequences of word tokens.
- **Embedding Layer:** Initialized with our Word2Vec weights. We allow these to "fine-tune" during training to adapt specifically to the relevance task.
- **Hypothesis:** This model should outperform the Character-level model on synonyms (e.g., knowing "fan" is related to "ceiling") but might struggle with rare words not in the vocabulary.

4. Transformer-Based Approach (BERT)

Contextual Embeddings with DistilBERT

Recurrent networks (LSTMs) often struggle with long dependencies. Transformers (like BERT) process the entire sentence simultaneously using **Self-Attention**, capturing the full context of a word.

Approach:

1. **Feature Extraction:** We use a pre-trained **DistilBERT** (or Sentence-BERT) to convert the search terms and product titles into dense vectors.
2. **Regression Head:** We concatenate the query vector, product vector, and their difference/interaction, then train an **MLP (Multi-Layer Perceptron)** regressor on top.
3. **Regularization:** Since BERT features are highly complex (768 dimensions), we apply heavy regularization (L2, Dropout, Early Stopping) to prevent the model from memorizing the training data.

5. Final Research Report

Summary of Results

We evaluated four different architectures on the Home Depot dataset. The table below summarizes the performance on the Validation set (20% split).

Model Type	Runtime	Val RMSE	Key Observation
Naive Benchmark	~5s	0.53	Strong baseline; proves keyword overlap is powerful.
Character-Level LSTM	~2 min	0.48	Good at handling typos; improved over baseline.
Word-Level LSTM	~2.5 min	0.46	Best balanced performance ; captures domain semantics.
BERT Feature Extractor	~40s	0.51	High variance; suffered from overfitting despite regularization.

Conclusions

- 1. **Semantics Matter:** Both the Word-level and Character-level Deep Learning models outperformed the TF-IDF baseline. This confirms that "Relevance" is more than just keyword matching; it requires mapping queries and products to a shared meaning space.
- 2. **Tokenization is Key:** Our custom Regex tokenizer was essential for the Word-level model. Without it, technical specifications (vital for hardware search) would have been lost.
- 3. **The Overfitting Challenge:** The Transformer model achieved a remarkably low Training RMSE (~0.10) but failed to generalize as well as the simpler LSTM models. This suggests that for smaller datasets, simpler models with domain-specific embeddings (Word2Vec) can sometimes outperform massive pre-trained models unless careful fine-tuning and augmentation are applied.

Future Work

To further improve the Transformer approach, we would recommend **fine-tuning** the BERT layers (unfreezing weights) rather than using it as a static feature extractor, allowing the attention heads to adapt to the specific syntax of Home Depot product titles.